

DPI343

Computer Organization and Assembly Language

Lecture 4.2

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

General Purpose Instructions.

Unconditional Jumping

JMP label

Transfers program execution to the address specified by the destination operand.

Loads EIP with the address specified (i.e. the next instruction to be executed by the CPU will be at this address).

This instruction is used to execute different types of jumps:

- **Near jump** - a jump to an instruction within the current code segment (the segment currently pointed to by the CS register. In FLAT memory model near jumps range from -2 to +2 gigabytes.
- **Short jump** - a near jump where the jump range is limited to -128 to +127 from the current EIP value
- **Far jump** - a jump to an instruction located in a different segment than the current code segment

If label is used to point to the instruction to jump to, then the compiler will calculate the value of the offset (i.e. distance in bytes) and will use the smallest possible JMP code (first Short, then Near, and then Far).

Instead of the label (i.e. memory address) the operand can also be the general purpose register or immediate value.

General Purpose Instructions.

Conditional Jumping

- Various conditional jump instructions are available, which allows to make decisions based on:
 - relationship between signed operands (greater-less)
 - relationship between unsigned operands (above-below)
 - values of arithmetic flags (ZF, SF, CF, OF, PF) in the flags register
- Common abbreviation of conditions:

Mnemonic	Meaning	Operand type
E	Equal	any
N	Not	any
G	Greater	signed
L	Less	signed
A	Above	unsigned
B	Below	unsigned

General Purpose Instructions.

Conditional jumping, cont.

- Conditional jump analyses bit(s) of the flags register and depending on the values of these bits transfers (or not) a program execution to a given label operand.

- The flags register is usually set by the compare instruction:

CMP operand1, operand2

- CMP works the same way as SUB instruction, except it does not save result.
- Flags can be analyzed using conditional jump instructions:
 - *Jcc label*
 - where *cc* is abbreviation of condition (see previous slide).
- Instead of CMP, you may use any instruction, which changes flags (e.g. arithmetic instructions) or the value of the ECX register.

CPU Execution Status

Flags Register *(see Lecture 2)*

- Flags register is a CPU register where bits represent characteristics of the results of operations and the state of the CPU
- Layout of flags register – some most important flags

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				OF				SF	ZF				PF		CF

CF	Carry	Carry or borrow to destination operand
PF	Parity	Even parity in low-order byte of result
ZF	Zero	Result equal to 0
SF	Sign	Result has negative sign
OF	Overflow	Result too large for destination

Instructions executed by CPU can change the values of flags depending on the result of the instruction. See manuals to find out which flags are affected by particular instruction.

Conditional jump instructions. Summary

Mnemonic	Criterion	Operand types	Flags Condition
JE / JZ	operand1 == operand2	any	ZF == 1
JNE / JNZ	operand1 <> operand2	any	ZF == 0
JL / JNGE	operand1 < operand2	signed	SF <> OF
JLE / JNG	operand1 <= operand2	signed	SF <> OF or ZF == 1
JG / JNLE	operand1 > operand2	signed	SF == OF and ZF == 0
JGE / JNL	operand1 >= operand2	signed	SF == OF
JB / JNAE / JC	operand1 < operand2	unsigned	CF == 1
JBE / JNA	operand1 <= operand2	unsigned	CF == 1 or ZF == 1
JA / JNBE	operand1 > operand2	unsigned	CF == 0 and ZF == 0
JAE / JNB / JNC	operand1 >= operand2	unsigned	CF == 0
JP	parity	any	PF == 1
JNP	no parity	any	PF == 0
JS	sign	any	SF == 1
JNS	no sign	any	SF == 0
JO	overflow	any	OF == 1
JNO	no overflow	any	OF == 0

Example 1

```
    :::  
    cmp eax, 100h ; compare EAX and value 10016 (25610)  
    jg  branch_b  ; jump to label branch_b if EAX > 10016  
  
    jl  branch_c  ; jump to label branch_c if EAX < 10016  
                    ; otherwise (i.e. EAX == 10016)  
  
    jmp finish    ; jump to finish
```

branch_b:

```
    :::  
    jmp finish    ; must jump to skip over branch_c !
```

branch_c:

```
    :::
```

finish:

```
    :::
```

Example 2

Be careful when checking signed and unsigned values!

```
.data
x      dword    -1
msgNeg byte     "Negative x", 0
msgPos byte     "Positive x", 0

.code
; . . .
mov     eax, x      ; copy value from memory word x to EAX
cmp     eax, 0      ; compare the content of EAX with 0
jb      negative    ; if below jump to label negative
positive:
push    offset msgPos
jmp     show
negative:
push    offset msgNeg
show:
call    puts        ; show message
```

Wrong conditional jump used!
jl should be used for signed values.
jb and ja are for unsigned values.

What will be output of this code – Positive or Negative?

Conditional jumping, cont.

JCXZ label

- Jump if CX is zero

JECXZ label

- Jump if ECX is zero
- The label must be in distance between -128 and +127 bytes from the instruction following the jump
- Useful to check value of the counter register before looping (loop instruction will be explained in the next lecture).

Bit operations.

Logic negation instruction - NOT

NOT operand

Logical negation of operand, e.g.:

NOT 11101010b (0EAh == -22 == 234)
00010101b (15h == 21)

NOT 11111111b (0FFh == -1)
00000000b (00h == 0)

Do not confuse this with the arithmetic negation instruction NEG!

NEG 11101010b (0EAh == -22)
00010110b (16h == 22)

NEG 11111111b (0FFh == -1)
00000001b (01h == 1)

Bit operations.

Logic conjunction – AND and TEST

AND operand1, operand2

Logical multiplication (conjunction) of two operands, e.g.:

```
    10101010b (0AAh)
AND 11000000b (0C0h)
-----
    10000000b ( 80h)
```

TEST operand1, operand2

Tests operands with logical multiplication. Does not change operand1, but sets ZF, SF and PF flags the same way as in **AND** instruction. Both **AND** and **TEST** instructions always clears OF and CF flags.

Often used

- to clear particular bit(s) of operand1, operand2 serving as a mask, e.g.:

```
mov    AL, 0AAh      ; 10101010b
and     AL, 01110111b ; clear 4th and 8th bits in register AL
                        ; AL == 22h (00100010b)
```
- to check values of particular bits

```
test    AL, 10000000b ; to check value of the high order bit
jnz     IsOne
```

Bit operations.

Logic disjunction - OR

OR operand1, operand2

Logical addition (disjunction) of operands:

	10101010b	(0AAh)
OR	<u>11000000b</u>	(0C0h)
	11101010b	(0EAh)

Instruction **OR** always sets SF, ZF and PF flags, and clears OF and CF flags according to the result.

Often used to set particular bit(s) of operand1 to 1, using operand2 as a mask, e.g.:

```
mov    AL, 0AAh        ; 10101010b
or     AL, 00101100b    ; set specified bits to 1 in the
                        ; register AL == 0AEh (10101110b)
```

Bit operations.

Logic exclusive disjunction - XOR

XOR operand1, operand2

Logical exclusive addition:

```
      10101010b (0AAh)
XOR  11000000b (0C0h)
      01101010b ( 6Ah)
```

Instruction always sets SF, ZF and PF flags, and clears OF and CF flags according to the result.

XOR examples

XOR is often used to clear the contents of a register (e.g., `xor eax, eax`) or to get differing bit(s) of both operands, or to invert bit(s) in operand1

```
xor  bx, bx    ; the same as mov bx, 0
                ; but also clears OF, SF and CF flags and sets ZF, PF
```

```
mov  al, 0AAh ; 10101010b
xor  al, 110b  ; inverts 2nd and 3rd bit of the register AL
                ; AL == 10101100b (ACh), clears ZF, sets SF and PF
```

```
mov  al, 0CDh ; 11001101b
mov  bl, 0ABh ; 10101011b
xor  al, bl    ; AL == 01100110b (66h), clears SF, ZF, sets PF
```

```
xor  eax, ebx ;
xor  ebx, eax ; } result is the same as xchg eax, ebx
xor  eax, ebx ; }
```

Bit operations.

Using for branching

Because **AND**, **OR**, **XOR** and **TEST** instructions affects flags ZF, SF and PF, this may be used for branching decisions, e.g.:

```
    ; :::  
    test    eax, 4    ; 4 == 0100b  
    jnz     Skip      ; jump to Skip if the 3rd bit in EAX is 1  
    ; :::  
Skip:  
    ; :::
```

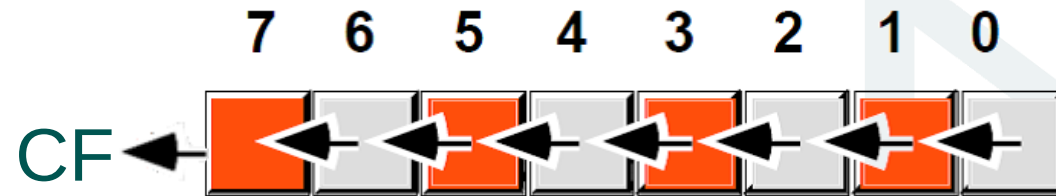
Remember:

The size of both operands always must be the same (byte, word, double word)

Bit operations.

Shift and Rotate

Shift Left (SHL/SAL)



Shift Right (SHR)



Arithmetic Shift Right (SAR)



Rotate Left (ROL/RCL)



Rotate Right (ROR/RCR)



Thank You!

